

SYNCHRONOUS FIFO

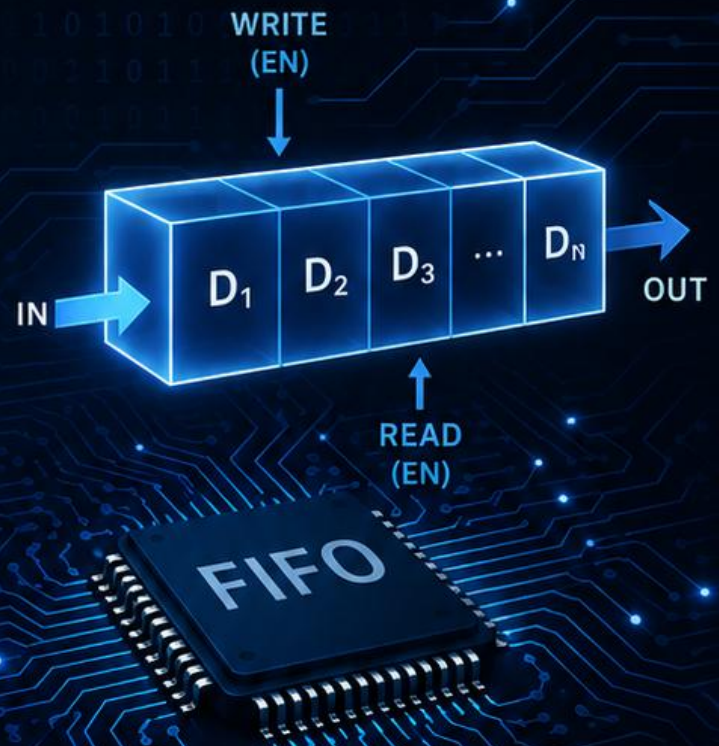
VERIFICATION PROJECT

USING UVM

FIFO

FIRST IN, FIRST OUT

DIGITAL ELECTRONICS PROJECT



Prepared by:
Mahmoud Abdelnasser Mahmoud



Supervised by:
Eng. Kareem Waseem

How UVM Works

The Universal Verification Methodology (UVM) provides a structured way to verify hardware designs using reusable and scalable components.

At the beginning, the **top module** generates the clock and instantiates both the **interface** and the **DUT** (FIFO).

The interface is then passed into the UVM environment using a **virtual interface**, which is stored inside the UVM configuration database.

During simulation, the **uvm_test** retrieves this virtual interface and uses it to control and monitor the DUT through the verification components.

UVM Architecture Components

Inside the test, an **environment (uvm_env)** is created. This environment contains:

- **Scoreboard** → checks correctness of outputs
- **Coverage Collector** → measures functional coverage
- **Agent** → responsible for driving and monitoring transactions

The **agent** includes:

- **Sequencer** → controls the flow of transactions
- **Driver** → converts transactions into signals
- **Monitor** → observes DUT behavior

UVM Data Flow

1. A **sequence** generates transactions called **sequence_items**.
2. These transactions are sent to the **sequencer**.
3. The **sequencer** forwards them to the **driver**.
4. The **driver** converts them into pin-level signals and drives the DUT through the **virtual interface**.
5. The **monitor** observes both the inputs and outputs of the DUT.

6. The monitor sends the collected data to:

- **Scoreboard** → for checking correctness
- **Coverage collector** → for measuring coverage

Scoreboard & Checking

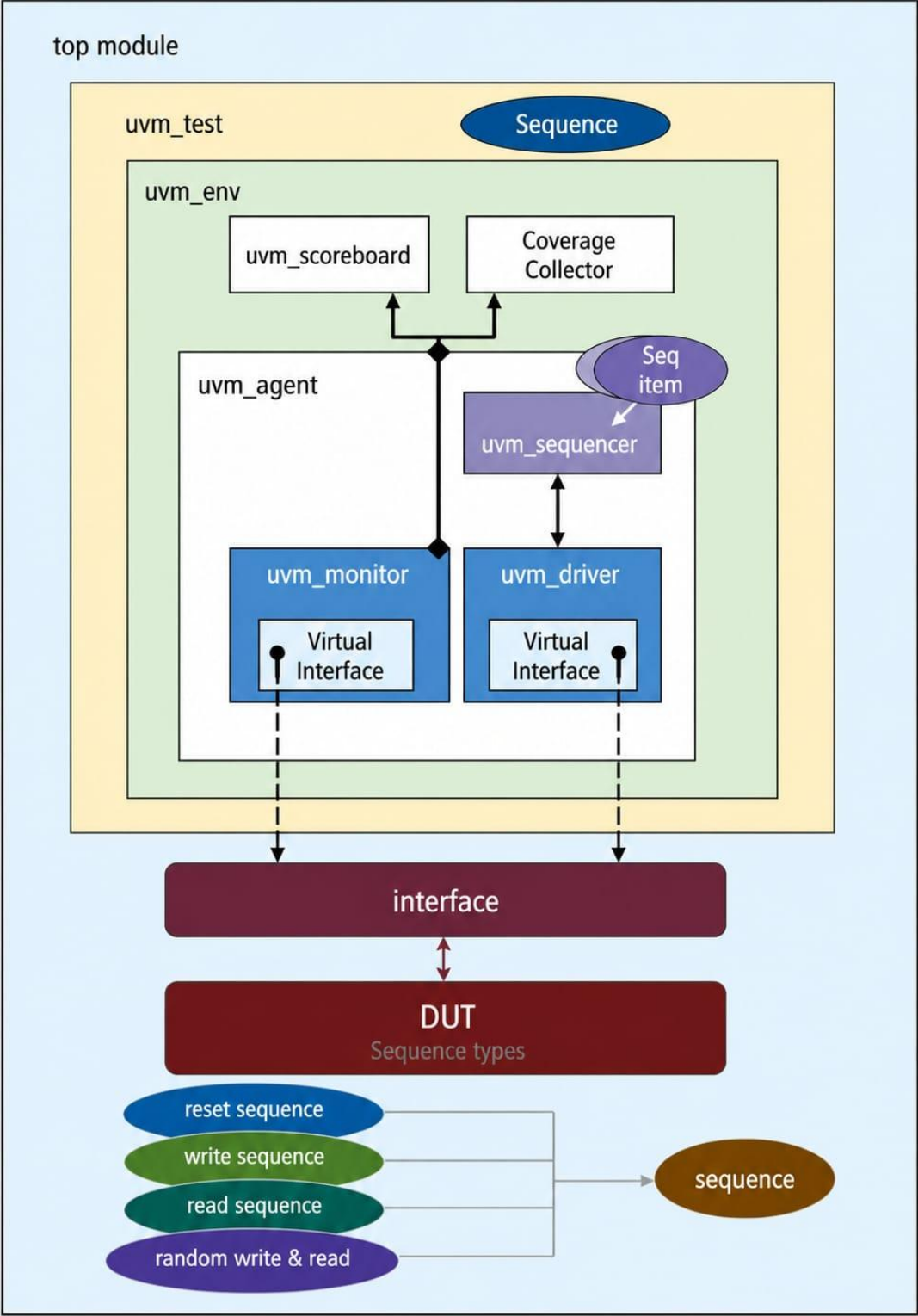
The **scoreboard** compares the DUT output with a **reference model** to ensure correctness of the design behavior.

At the same time, **assertions (SVA)** can be bound to the DUT to verify protocol correctness and catch design violations early.

Execution Flow Summary

- uvm_test starts simulation
- Builds the uvm_env
- Runs sequences (stimulus generation)
- Driver applies stimulus to DUT
- Monitor captures results
- Scoreboard checks correctness
- Coverage tracks verification completeness

UVM testbench structure



Verification Plan :

	Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
1					
2	FIFO_1	During reset, all status signals (full, almostfull, overflow, underflow) must be low, while empty is asserted	Directed reset at start of simulation, then random traffic after reset	Cover reset scenarios	Assertion + scoreboard validation for reset behavior
3	FIFO_2	After reset, when wr_en is active and FIFO is not full, data is written and write pointer increments, wr_ack should be asserted	Constrained random with wr_en active under non-full condition	Cover all combinations of wr_en and wr_ack	Assertion + scoreboard validation for write operation
4	FIFO_3	When rd_en is active and FIFO is not empty, data_out should match stored data at read pointer	Constrained random with rd_en active under non-empty condition	Cover all rd_en scenarios	Scoreboard validation for data_out correctness
5	FIFO_4	When FIFO becomes full, full flag must be asserted	Constrained random reaching full condition	Cover full flag activation	Assertion + scoreboard validation for full flag
6	FIFO_5	When FIFO is almost full (one slot left), almostfull flag must be asserted	Constrained random near full boundary	Cover almostfull scenarios	Assertion + scoreboard validation for almostfull
7	FIFO_6	When FIFO becomes empty, empty flag must be asserted	Constrained random reaching empty condition	Cover empty scenarios	Assertion + scoreboard validation for empty flag
8	FIFO_7	When FIFO has one element left, almostempty flag must be asserted	Constrained random near empty boundary	Cover almostempty scenarios	Assertion + scoreboard validation for almostempty
9	FIFO_8	When FIFO is full and wr_en is active, overflow must be asserted	Constrained random with wr_en under full condition	Cover overflow cases	Assertion + scoreboard validation for overflow
10	FIFO_9	When FIFO is empty and rd_en is active, underflow must be asserted	Constrained random with rd_en under empty condition	Cover underflow cases	Assertion + scoreboard validation for underflow
11	FIFO_10	When wr_en is active and FIFO is not full, count and write pointer should increment	Constrained random write operations	Cover write pointer increment cases	Concurrent assertion for wr_ptr and count
12	FIFO_11	When rd_en is active and FIFO is not empty, count decrements and read pointer increments	Constrained random read operations	Cover read pointer increment cases	Concurrent assertion for rd_ptr and count

UVM Codes :

Interface :

```
interface FIFO_if(clk);
    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 8 ;
    input bit clk ;
    logic [FIFO_WIDTH-1:0] data_in;
    logic rst_n, wr_en, rd_en;
    logic [FIFO_WIDTH-1:0] data_out;
    logic wr_ack, overflow;
    logic full, empty, almostfull, almostempty, underflow;

    modport DUT (
        input clk ,rst_n,wr_en,rd_en,data_in,
        output data_out , wr_ack,overflow, full , empty, almostfull, almostempty, underflow
    );
    modport TEST (
        input clk ,data_out,wr_ack,overflow,full,empty,almostfull,almostempty,underflow,
        output rst_n, wr_en , rd_en,data_in
    );
    modport MONITOR (input clk, rst_n, wr_en, rd_en , data_in, data_out, wr_ack,
        overflow, full, empty, almostfull, almostempty, underflow);
endinterface
```

Design after bugs corrected :

```
module FIFO(FIFO_if.DUT fifoif);
parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 8;
localparam max_fifo_addr = $clog2(FIFO_DEPTH);
reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];
reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
reg [max_fifo_addr:0] count;

always @(posedge fifoif.clk or negedge fifoif.rst_n) begin
    if (!fifoif.rst_n) begin
        wr_ptr <= 0;
        fifoif.wr_ack <= 0; //missing after reset
        fifoif.overflow <= 0; //missing after reset
    end
    else if (fifoif.wr_en && count < FIFO_DEPTH) begin
        mem[wr_ptr] <= fifoif.data_in;
        fifoif.wr_ack <= 1;
        wr_ptr <= wr_ptr + 1;
    end
    else begin
        fifoif.wr_ack <= 0;
        if (fifoif.full && fifoif.wr_en) // & must be &&
            fifoif.overflow <= 1;
        else
            fifoif.overflow <= 0;
    end
end

always @(posedge fifoif.clk or negedge fifoif.rst_n) begin
    if (!fifoif.rst_n) begin
        rd_ptr <= 0;
        fifoif.underflow <= 0; //missing after reset
    end
    else if (fifoif.rd_en && count != 0) begin
        fifoif.data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;
    end
    else begin
        if(fifoif.empty && fifoif.rd_en)begin
            fifoif.underflow <= 1;
        end
        else begin
            fifoif.underflow <= 0;
        end
    end
end
end
```



```

always @(posedge fifoif.clk or negedge fifoif.rst_n) begin
    if (!fifoif.rst_n) begin
        count <= 0;
    end
    else begin
        if ( ({fifoif.wr_en, fifoif.rd_en} == 2'b10) && !fifoif.full)
            count <= count + 1;
        else if ( ({fifoif.wr_en, fifoif.rd_en} == 2'b01) && !fifoif.empty)
            count <= count - 1;
        else if ( ({fifoif.wr_en, fifoif.rd_en} == 2'b11) && fifoif.empty) //missing case
            count <= count + 1;
        else if ( ({fifoif.wr_en, fifoif.rd_en} == 2'b11) && fifoif.full)
            count <= count - 1;
    end
end

assign fifoif.full = (count == FIFO_DEPTH)? 1 : 0;
assign fifoif.empty = (count == 0)? 1 : 0;
//assign fifoif.underflow = (fifoif.empty && fifoif.rd_en)? 1 : 0;    // its mentioned
in pdf its a sequential

assign fifoif.almostfull = (count == FIFO_DEPTH-1)? 1 : 0; //bug in -2
assign fifoif.almostempty = (count == 1)? 1 : 0;

endmodule

```

assertions

```

module FIFO_sva (FIFO_if fifoif);

parameter FIFO_DEPTH = 8;

// RESET CHECK

property reset_p;
    @(posedge fifoif.clk)
    !fifoif.rst_n | =>
        (fifoif.wr_ack == 0 &&
         fifoif.overflow == 0 &&
         fifoif.underflow == 0);
endproperty

reset_ap: assert property (reset_p);
reset_cp: cover property (reset_p);

// WRITE ACK CHECK

```

```

property wr_ack_p;
    @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
        (!fifoif.full && fifoif.wr_en) | => fifoif.wr_ack;
endproperty

wr_ack_ap: assert property (wr_ack_p);
wr_ack_cp: cover property (wr_ack_p);


// OVERFLOW CHECK

property overflow_p;
    @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
        (fifoif.full && fifoif.wr_en) | => fifoif.overflow;
endproperty

overflow_ap: assert property (overflow_p);
overflow_cp: cover property (overflow_p);


// UNDERFLOW CHECK

property underflow_p;
    @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
        (fifoif.empty && fifoif.rd_en) | => fifoif.underflow;
endproperty

underflow_ap: assert property (underflow_p);
underflow_cp: cover property (underflow_p);


// FULL FLAG CHECK

property full_flag_p;
    @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
        fifoif.full | -> fifoif.full;
endproperty

full_flag_ap: assert property (full_flag_p);


// EMPTY FLAG CHECK

property empty_flag_p;
    @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
        fifoif.empty | -> fifoif.empty;
endproperty

```



```

empty_flag_ap: assert property (empty_flag_p);

// ALMOST FULL CHECK

property almostfull_p;
    @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
        fifoif.almostfull |-> fifoif.almostfull;
endproperty

almostfull_ap: assert property (almostfull_p);

// ALMOST EMPTY CHECK

property almostempty_p;
    @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
        fifoif.almostempty |-> fifoif.almostempty;
endproperty

almostempty_ap: assert property (almostempty_p);

// VALID READ DATA CHECK

property read_valid_p;
    @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
        (fifoif.rd_en && !fifoif.empty) |=> !$isunknown(fifoif.data_out);
endproperty

read_valid_ap: assert property (read_valid_p);

// VALID WRITE CHECK

property write_valid_p;
    @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
        (fifoif.wr_en && !fifoif.full) |=> fifoif.wr_ack;
endproperty

write_valid_ap: assert property (write_valid_p);

endmodule

```

Top :

```

import uvm_pkg::*;
`include "uvm_macros.svh"
import fifo_test_pkg::*;

```

```

module top();

    bit clk;
    initial begin
        forever
            #1 clk = ~clk;
    end
    FIFO_if fifoif (clk);

    FIFO DUT (fifoif);

bind FIFO FIFO_sva svas (.fifoif(fifoif));

    initial begin
        uvm_config_db #(virtual FIFO_if)::set (null, "uvm_test_top", "FIFO_if", fifoif);
        run_test("fifo_test");
    end

endmodule

```

scoreboard:

```

package fifo_scoreboard_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"
import fifo_seq_item_pkg::*;

class fifo_scoreboard extends uvm_scoreboard;

    `uvm_component_utils(fifo_scoreboard)

    uvm_analysis_export #(fifo_seq_item) sb_export;
    uvm_tlm_analysis_fifo #(fifo_seq_item) sb_fifo;

    fifo_seq_item tr;

    bit [FIFO_WIDTH-1:0] ref_data_out;
    bit ref_wr_ack, ref_overflow;
    bit ref_full, ref_empty, ref_almostfull, ref_almostempty, ref_underflow;

    bit [FIFO_WIDTH-1:0] ref_mem [$];

    int error_cnt = 0;
    int correct_cnt = 0;

    function new(string name="fifo_scoreboard", uvm_component parent=null);
        super.new(name,parent);
    endfunction

```

```

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    sb_export = new("sb_export", this);
    sb_fifo   = new("sb_fifo", this);
endfunction

function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    sb_export.connect(sb_fifo.analysis_export);
endfunction

task run_phase(uvm_phase phase);
    super.run_phase(phase);

    forever begin
        sb_fifo.get(tr);
        ref_model(tr);

        if(tr.data_out != ref_data_out) begin
            error_cnt++;
            `uvm_info("run_phase", $sformatf(
                "error %s , out_ref:%0h , wr_ack_ref:%0b, overflow_ref:%0b,
underflow_ref:%0b, full_ref:%0b, empty_ref:%0b, almostfull_ref:%0b, almostempty_ref:%0b",
                tr.convert2string(), ref_data_out, ref_wr_ack, ref_overflow,
ref_underflow, ref_full, ref_empty, ref_almostfull, ref_almostempty
            ), UVM_MEDIUM);
        end
        else begin
            correct_cnt++;
        end
    end
endtask

task ref_model(fifo_seq_item tr);

    if(!tr.rst_n) begin
        ref_wr_ack = 0;
        ref_overflow = 0;
        ref_underflow = 0;
        ref_full = 0;
        ref_empty = 1;
        ref_almostempty = 0;
        ref_almostfull = 0;
        ref_mem.delete();
    end
    else begin

        if(tr.wr_en && tr.rd_en && (ref_mem.size()==0 || ref_mem.size()==8)) begin

            if(ref_mem.size()==0) begin

```

```

        ref_mem.push_back(tr.data_in);
        ref_wr_ack = 1;
        ref_overflow = 0;
        ref_underflow = 0;
    end
    else if(ref_mem.size()==8) begin
        ref_data_out = ref_mem.pop_front();
        ref_underflow = 0;
        ref_underflow = 0;
    end

end

else begin

    if(tr.wr_en) begin
        ref_underflow = 0;
        if(ref_mem.size() < 8) begin
            ref_mem.push_back(tr.data_in);
            ref_wr_ack = 1;
            ref_overflow = 0;
        end
        else begin
            ref_wr_ack = 0;
            ref_overflow = 1;
        end
    end

    if(tr.rd_en) begin
        ref_overflow = 0;
        if(ref_mem.size() != 0) begin
            ref_data_out = ref_mem.pop_front();
            ref_underflow = 0;
        end
        else begin
            ref_underflow = 1;
        end
    end

end

end

end

if(ref_mem.size() == FIFO_DEPTH)
    ref_full = 1;
else
    ref_full = 0;

if(ref_mem.size() == 0)
    ref_empty = 1;
else
    ref_empty = 0;

```

```

        if(ref_mem.size() == FIFO_DEPTH-1)
            ref_almostfull = 1;
        else
            ref_almostfull = 0;

        if(ref_mem.size() == 1)
            ref_almostempty = 1;
        else
            ref_almostempty = 0;

    endtask

    function void report_phase(uvm_phase phase);
        super.report_phase(phase);

        `uvm_info("report_phase", $sformatf("correct: %0d", correct_cnt), UVM_MEDIUM);
        `uvm_info("report_phase", $sformatf("error: %0d", error_cnt), UVM_MEDIUM);
    endfunction

endclass

endpackage

```

coverage package :

```

package fifo_coverage_pkg;

import fifo_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class fifo_coverage extends uvm_component;
    `uvm_component_utils(fifo_coverage);

    uvm_analysis_export #(fifo_seq_item) cov_export;
    uvm_tlm_analysis_fifo #(fifo_seq_item) cov_fifo;
    fifo_seq_item seq_item_cov;

    //coverage group

    covergroup FIFOcg ;

        wr_en_cp: coverpoint seq_item_cov.wr_en;
        rd_en_cp: coverpoint seq_item_cov.rd_en;
        full_cp: coverpoint seq_item_cov.full;
        almostfull_cp: coverpoint seq_item_cov.almostfull;
        empty_cp: coverpoint seq_item_cov.empty;
        almostempty_cp: coverpoint seq_item_cov.almostempty;
        overflow_cp: coverpoint seq_item_cov.overflow;
        underflow_cp: coverpoint seq_item_cov.underflow;
        wr_ack_cp: coverpoint seq_item_cov.wr_ack;
    endcovergroup
endclass

```

```

//ignored when rd_en =1 with full not important
full_cross: cross wr_en_cp, rd_en_cp, full_cp{
    ignore_bins rd_full = binsof(rd_en_cp)intersect{1} &&
    binsof(full_cp)intersect{1}
;}

almostfull_cross: cross wr_en_cp, rd_en_cp, almostfull_cp;
empty_cross: cross wr_en_cp, rd_en_cp, empty_cp;
almostempty_cross: cross wr_en_cp, rd_en_cp, almostempty_cp;

//ignored when wr_en =0 and overflow =1 not important
overflow_cross: cross wr_en_cp, rd_en_cp, overflow_cp{
    ignore_bins not_wr_overflow = binsof(wr_en_cp)intersect{0} &&
    binsof(overflow_cp)intersect{1}
;}

//ignored when rd_en =0 and underflow =1 not important
underflow_cross: cross wr_en_cp, rd_en_cp, underflow_cp{
    ignore_bins not_rd_overflow = binsof(rd_en_cp)intersect{0} &&
    binsof(underflow_cp)intersect{1}
;}

//ignored when wr_en=0 with wr_ack =1 not important
wr_ack_cross: cross wr_en_cp, rd_en_cp, wr_ack_cp{
    ignore_bins not_wren_wrack = binsof(wr_en_cp)intersect{0} &&
    binsof(wr_ack_cp)intersect{1}
;}

endgroup

function new(string name="fifo_coverage", uvm_component parent);
    super.new(name, parent);
    FIFOcg = new();
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    cov_export = new("cov_export", this);
    cov_fifo = new("cov_fifo", this);
endfunction

function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    cov_export.connect(cov_fifo.analysis_export);
endfunction

task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
        cov_fifo.get(seq_item_cov);
        FIFOcg.sample();
    end
endtask

```

```
endclass
```

```
endpackage
```

agent:

```
package fifo_agent_pkg;
```

```
import fifo_cfg_pkg::*;
```

```
import fifo_driver_pkg::*;
```

```
import fifo_sequencer_pkg::*;
```

```
import fifo_monitor_pkg::*;
```

```
import fifo_seq_item_pkg::*;
```

```
import uvm_pkg::*;
```

```
`include "uvm_macros.svh"
```

```
class fifo_agent extends uvm_agent;
```

```
`uvm_component_utils(fifo_agent);
```

```
fifo_driver driver;
```

```
fifo_sequencer sqr;
```

```
fifo_monitor mon;
```

```
fifo_config fifo_cfg;
```

```
uvm_analysis_port #(fifo_seq_item) agt_ap;
```

```
function new(string name="fifo_agent", uvm_component parent = null);
```

```
    super.new(name, parent);
```

```
endfunction
```

```
function void build_phase (uvm_phase phase);
```

```
    super.build_phase(phase);
```

```
    driver = fifo_driver::type_id::create("fifo_driver", this);
```

```
    sqr = fifo_sequencer::type_id::create("fifo_sequencer", this);
```

```
    mon = fifo_monitor::type_id::create("fifo_monitor", this);
```

```
    fifo_cfg = fifo_config::type_id::create("fifo_config");
```

```
    agt_ap = new("agt_ap", this);
```

```
    if(!uvm_config_db #(fifo_config) :: get(this, "", "CFG", fifo_cfg))begin  
        `uvm_fatal("build_phase", "Unable to get configuration object")  
    end
```

```
end
```

```
endfunction
```

```
function void connect_phase(uvm_phase phase);
```

```
    super.connect_phase(phase);
```

```
    driver.FIFO_vif = fifo_cfg.FIFO_vif;
```

```
    mon.FIFO_vif = fifo_cfg.FIFO_vif;
```

```
    driver.seq_item_port.connect(sqr.seq_item_export);
```

```
    mon.mon_ap.connect(agt_ap);
```



```
endfunction
```

```
endclass
```

```
endpackage
```

config:

```
package fifo_cfg_pkg;
```

```
import uvm_pkg::*;
```

```
`include "uvm_macros.svh"
```

```
class fifo_config extends uvm_object;
```

```
`uvm_object_utils(fifo_config)
```

```
virtual FIFO_if FIFO_vif;
```

```
function new(string name = "fifo_config");
```

```
    super.new(name);
```

```
endfunction
```

```
endclass
```

```
endpackage
```

driver:

```
package fifo_driver_pkg;
```

```
import fifo_seq_item_pkg::*;
```

```
import uvm_pkg::*;
```

```
`include "uvm_macros.svh"
```

```
class fifo_driver extends uvm_driver #(fifo_seq_item);
```

```
`uvm_component_utils(fifo_driver)
```

```
virtual fifo_interface fifo_vif;
```

```
fifo_seq_item seq_item;
```

```
function new(string name="fifo_driver", uvm_component parent=null);
```

```
    super.new(name, parent);
```

```
endfunction
```

```
task run_phase(uvm_phase phase);
```

```
    super.run_phase(phase);
```

```
    forever begin
```

```
        seq_item = fifo_seq_item::type_id::create("fifo_seq_item");
```

```
        seq_item_port.get_next_item(seq_item);
```

```

        fifo_vif.data_in      = seq_item.data_in;
        fifo_vif.rst_n       = seq_item.rst_n;
        fifo_vif.wr_en       = seq_item.wr_en;
        fifo_vif.rd_en       = seq_item.rd_en;
        @(negedge fifo_vif.clk);
        seq_item_port.item_done();
        `uvm_info("run_phase", seq_item.convert2string, UVM_MEDIUM)
    end
endtask

endclass

endpackage

```

env:

```

package fifo_env_pkg;
import fifo_driver_pkg::*;
import fifo_coverage_pkg::*;
import fifo_scoreboard_pkg::*;
import fifo_agent_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class fifo_env extends uvm_component;
    `uvm_component_utils(fifo_env)

    fifo_agent agt;
    fifo_scoreboard sb;
    fifo_coverage cov;

    function new (string name = "fifo_env", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        agt = fifo_agent::type_id::create("agt", this);
        sb = fifo_scoreboard::type_id::create("sb", this);
        cov = fifo_coverage::type_id::create("cov", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        agt.agt_ap.connect(sb.sb_export);
        agt.agt_ap.connect(cov.cov_export);
    endfunction

```

```
endclass  
endpackage
```

monitor :

```
package fifo_monitor_pkg;  
`include "uvm_macros.svh"  
import fifo_seq_item_pkg::*;  
import uvm_pkg::*;  
  
class fifo_monitor extends uvm_monitor;  
    `uvm_component_utils(fifo_monitor)  
  
    virtual FIFO_if FIFO_vif;  
    uvm_analysis_port #(fifo_seq_item) mon_ap;  
    fifo_seq_item rsp_seq_item;  
  
    function new(string name = "fifo_monitor", uvm_component parent = null);  
        super.new(name, parent);  
    endfunction  
  
    function void build_phase(uvm_phase phase);  
        super.build_phase(phase);  
        mon_ap = new("mon_ap", this);  
    endfunction: build_phase  
  
    task run_phase(uvm_phase phase);  
        super.run_phase(phase);  
        forever begin  
            rsp_seq_item = fifo_seq_item::type_id::create("rsp_seq_item");  
            @(negedge FIFO_vif.clk);  
            rsp_seq_item.data_in = FIFO_vif.data_in;  
            rsp_seq_item.rst_n = FIFO_vif.rst_n;  
            rsp_seq_item.wr_en = FIFO_vif.wr_en;  
            rsp_seq_item.rd_en = FIFO_vif.rd_en;  
            rsp_seq_item.data_out = FIFO_vif.data_out;  
            rsp_seq_item.wr_ack = FIFO_vif.wr_ack;  
            rsp_seq_item.overflow = FIFO_vif.overflow;  
            rsp_seq_item.full = FIFO_vif.full;  
            rsp_seq_item.empty = FIFO_vif.empty;  
            rsp_seq_item.almostfull = FIFO_vif.almostfull;  
            rsp_seq_item.almostempty = FIFO_vif.almostempty;  
            rsp_seq_item.underflow = FIFO_vif.underflow;  
            mon_ap.write(rsp_seq_item);  
            `uvm_info("run_phase", rsp_seq_item.convert2string(), UVM_HIGH)  
        end  
    endtask  
  
endclass  
endpackage
```

read only seq :

```
package fifo_read_seq_pkg;
```

```

import fifo_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class fifo_read_seq extends uvm_sequence #(fifo_seq_item);
`uvm_object_utils(fifo_read_seq);
fifo_seq_item seq_item;

function new(string name = "fifo_read_seq");
    super.new(name);
endfunction

task body;
    seq_item = fifo_seq_item::type_id::create("seq_item");
    repeat(5000)begin
        start_item(seq_item);
        assert(seq_item.randomize());
        seq_item.wr_en = 0;
        seq_item.rd_en = 1;
        finish_item(seq_item);
    end
endtask
endclass

endpackage

```

read_write seq :

```

package fifo_write_read_seq_pkg;
import fifo_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class fifo_write_read_seq extends uvm_sequence #(fifo_seq_item);
`uvm_object_utils(fifo_write_read_seq);
fifo_seq_item seq_item;

function new(string name = "fifo_write_read_seq");
    super.new(name);
endfunction

task body;
    seq_item = fifo_seq_item::type_id::create("seq_item");
    repeat(5000)begin
        start_item(seq_item);
        assert(seq_item.randomize());
        finish_item(seq_item);
    end
endtask
endclass

endpackage

```

write only seq:

```
package fifo_write_seq_pkg;
import fifo_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class fifo_write_seq extends uvm_sequence #(fifo_seq_item);
`uvm_object_utils(fifo_write_seq);
fifo_seq_item seq_item;

function new(string name = "fifo_write_seq");
    super.new(name);
endfunction

task body;
    seq_item = fifo_seq_item::type_id::create("seq_item");
    repeat(5000)begin
        start_item(seq_item);
        assert(seq_item.randomize());
        seq_item.wr_en = 1;
        seq_item.rd_en = 0;
        finish_item(seq_item);
    end
endtask
endclass

endpackage
```

reset seq:

```
package fifo_reset_seq_pkg;
import fifo_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class fifo_reset_seq extends uvm_sequence #(fifo_seq_item);
`uvm_object_utils(fifo_reset_seq);
fifo_seq_item seq_item;

function new(string name = "fifo_reset_seq");
    super.new(name);
endfunction

task body;
    seq_item = fifo_seq_item::type_id::create("seq_item");
    start_item(seq_item);
    seq_item.data_in = 0;
    seq_item.wr_ack = 0;
    seq_item.rd_en = 0;
    seq_item.rst_n = 0;
    finish_item(seq_item);
endtask
```

```
endclass

endpackage
```

seq item :

```
package fifo_seq_item_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"
parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 8;

class fifo_seq_item extends uvm_sequence_item;
`uvm_object_utils(fifo_seq_item);

rand logic [FIFO_WIDTH-1:0] data_in;
rand logic rst_n, wr_en, rd_en;
logic [FIFO_WIDTH-1:0] data_out;
logic wr_ack, overflow;
logic full, empty, almostfull, almostempty, underflow;

int RD_EN_ON_DIST, WR_EN_ON_DIST;

function new (string name = "fifo_seq_item", int RD_EN_ON_DIST_params=30, int
WR_EN_ON_DIST_params=70);
    super.new(name);

    RD_EN_ON_DIST = RD_EN_ON_DIST_params;
    WR_EN_ON_DIST = WR_EN_ON_DIST_params;
endfunction

constraint Reset_con {
    rst_n dist {1:/90, 0:/10};
}

constraint Wr_en_con{
    wr_en dist {1:/WR_EN_ON_DIST, 0:/(100-WR_EN_ON_DIST)};
}

constraint Rd_en_con{
    rd_en dist {1:/RD_EN_ON_DIST, 0:/(100-RD_EN_ON_DIST)};
}

function string convert2string();
    return $sformatf("rst_n = %0d, wr_en = %0d, rd_en = %0d, data_in = %0d, data_out =
%0d, wr_ack = %0d, overflow = %0d, full = %0d, empty = %0d, almostfull = %0d, almostempty
= %0d, underflow = %0d",
```

```

        rst_n, wr_en, rd_en, data_in, data_out, wr_ack, overflow, full,
empty, almostfull, almostempty, underflow);
endfunction

endclass

endpackage

```

sequencer:

```

package fifo_sequencer_pkg;

import fifo_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class fifo_sequencer extends uvm_sequencer #(fifo_seq_item);
`uvm_component_utils(fifo_sequencer);

function new(string name="fifo_sequencer", uvm_component parent = null);
    super.new(name, parent);
endfunction

endclass

endpackage

```

Test:

```

package fifo_test_pkg ;

import fifo_cfg_pkg::*;
import fifo_env_pkg::*;
import uvm_pkg::*;
import fifo_write_seq_pkg::*;
import fifo_read_seq_pkg::*;
import fifo_write_read_seq_pkg::*;
import fifo_reset_seq_pkg::*;
`include "uvm_macros.svh"

class fifo_test extends uvm_test;
`uvm_component_utils(fifo_test)

fifo_env env;
fifo_config fifo_cfg;
fifo_reset_seq reset_seq;
fifo_write_seq write_seq;
fifo_read_seq read_seq;
fifo_write_read_seq write_read_seq;
virtual FIFO_if FIFO_vif;

```

```

function new (string name = "fifo_test", uvm_component parent = null);
    super.new(name, parent);
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);

    env = fifo_env::type_id::create("env", this);
    fifo_cfg = fifo_config::type_id::create("fifo_cfg");
    reset_seq = fifo_reset_seq::type_id::create("fifo_reset_seq");
    write_seq = fifo_write_seq::type_id::create("fifo_write_seq");
    read_seq = fifo_read_seq::type_id::create("fifo_read_seq");
    write_read_seq = fifo_write_read_seq::type_id::create("fifo_write_read_seq");

    if(!uvm_config_db #(virtual FIFO_if)::get(this, "", "FIFO_if", fifo_cfg.FIFO_vif))
        `uvm_fatal("build phase", "Cannot get the virtual interface");

    uvm_config_db #(fifo_config) :: set (this, "*", "CFG", fifo_cfg);
endfunction

task run_phase(uvm_phase phase);
    super.run_phase(phase);

    phase.raise_objection(this);
    `uvm_info("run_phase", "reset_seq Asserted", UVM_MEDIUM)
    reset_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "reset_seq deAsserted", UVM_MEDIUM)

    `uvm_info("run_phase", "write_seq Asserted", UVM_MEDIUM)
    write_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "write_seq deAsserted", UVM_MEDIUM)

    `uvm_info("run_phase", "read_seq Asserted", UVM_MEDIUM)
    read_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "read_seq deAsserted", UVM_MEDIUM)

    `uvm_info("run_phase", "write_read_seq Asserted", UVM_MEDIUM)
    write_read_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "write_read_seq deAsserted", UVM_MEDIUM)

    phase.drop_objection(this);
endtask

endclass

endpackage

```


do file :

```
run.do
1  vlib work
2  vlog -f src_files.list +cover -covercells
3  vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all -cover
4  coverage save fifo.ucdb -onexit
5  add wave /top/fifoif/*
6  run -all
```

List:

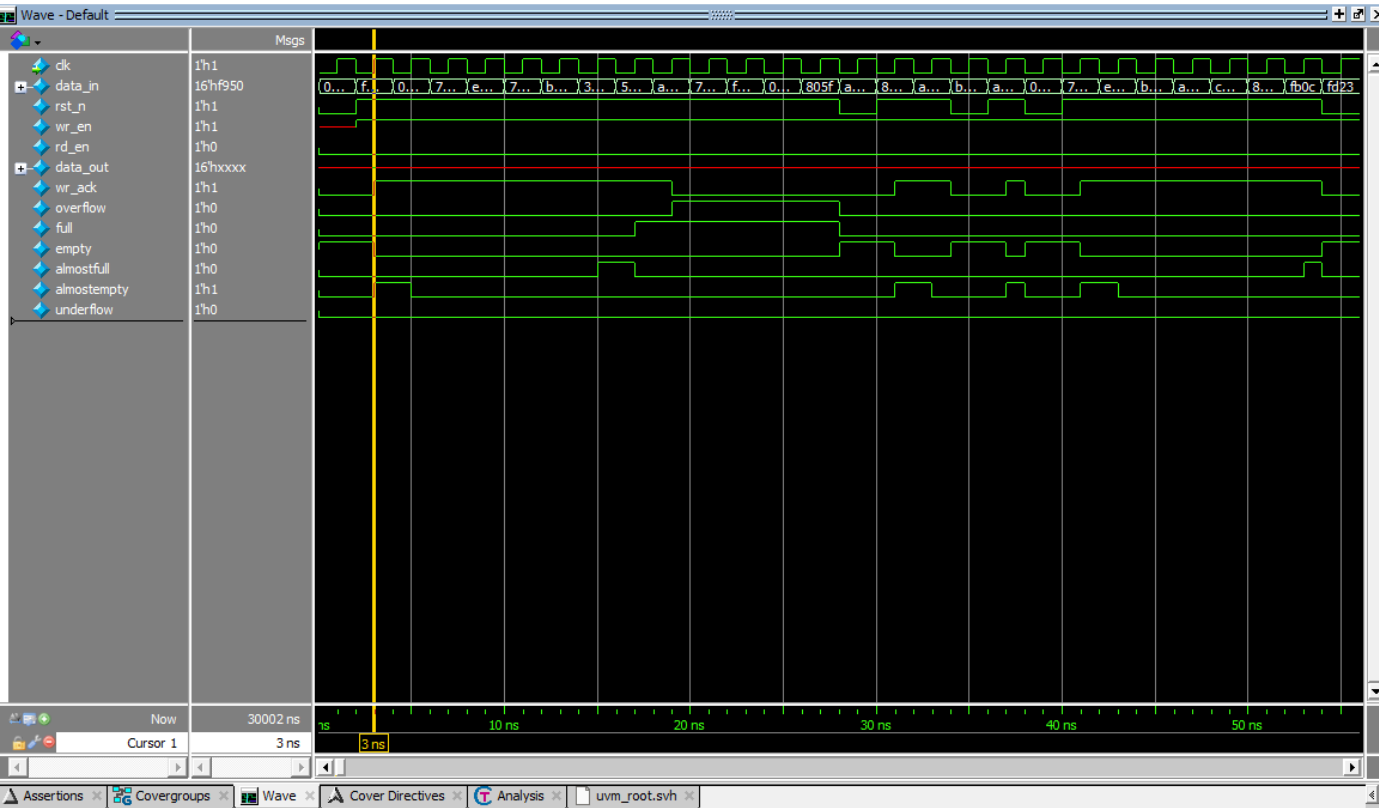
```
1  FIFO.sv
2  FIFO_sva.sv
3  FIFO_if.sv
4  seq_item.sv
5  config.sv
6  monitor.sv
7  driver.sv
8  sequencer.sv
9  reset_seq.sv
10 write_seq.sv
11 read_seq.sv
12 read_write_seq.sv
13 agent.sv
14 scoreboard.sv
15 coverage.sv
16 fifo_env.sv
17 FIFO_test.sv
18 FIFO_top.sv
```

Transcript showing zero error count and zero UVM_FATAL and zero UVM_ERROR:

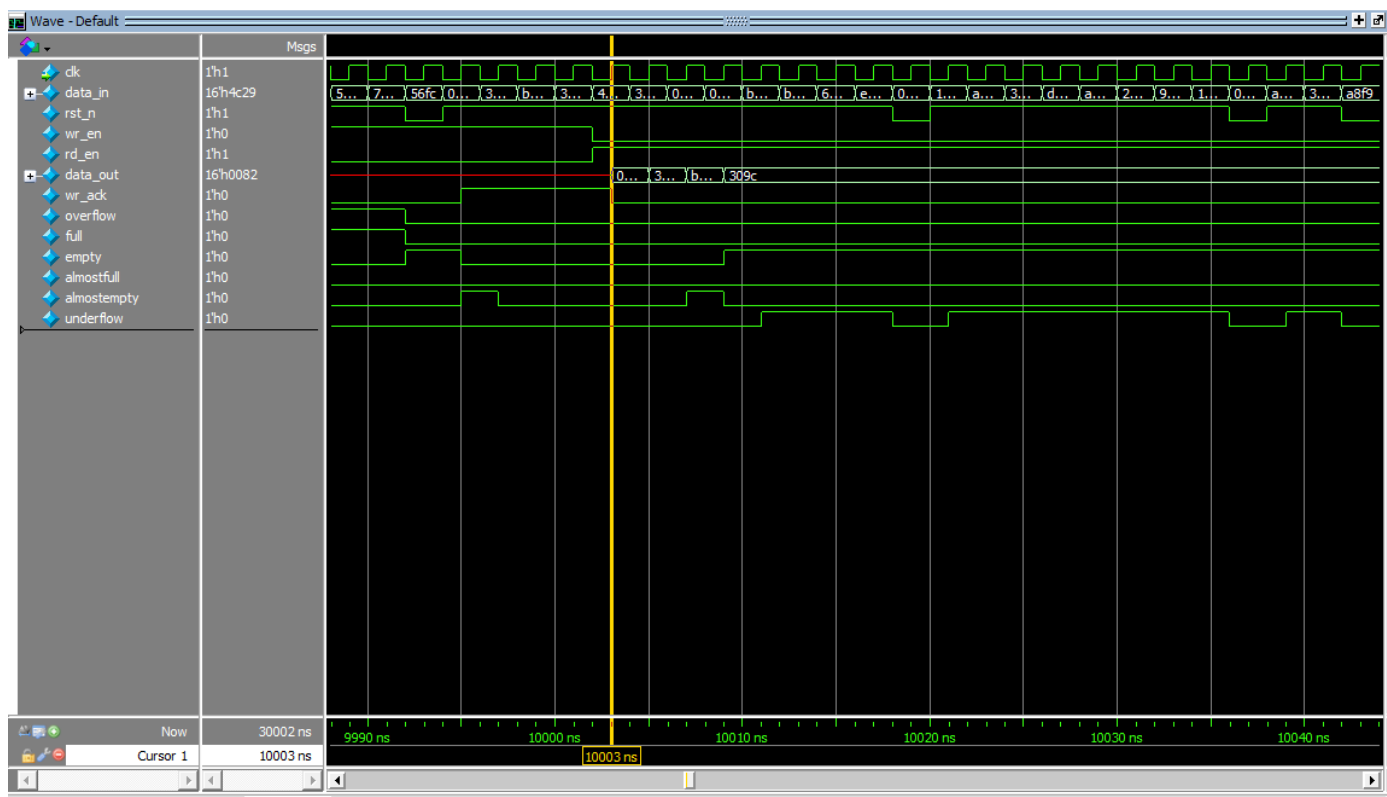
```
# UVM_INFO scoreboard.sv(154) @ 30002: uvm_test_top.env.sb [report_phase] correct: 15001
# UVM_INFO scoreboard.sv(155) @ 30002: uvm_test_top.env.sb [report_phase] error: 0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 14
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [report_phase] 2
# [run_phase] 8
# ** Note: $finish : C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 30002 ns Iteration: 61 Instance: /top
```

Questasim waveform :

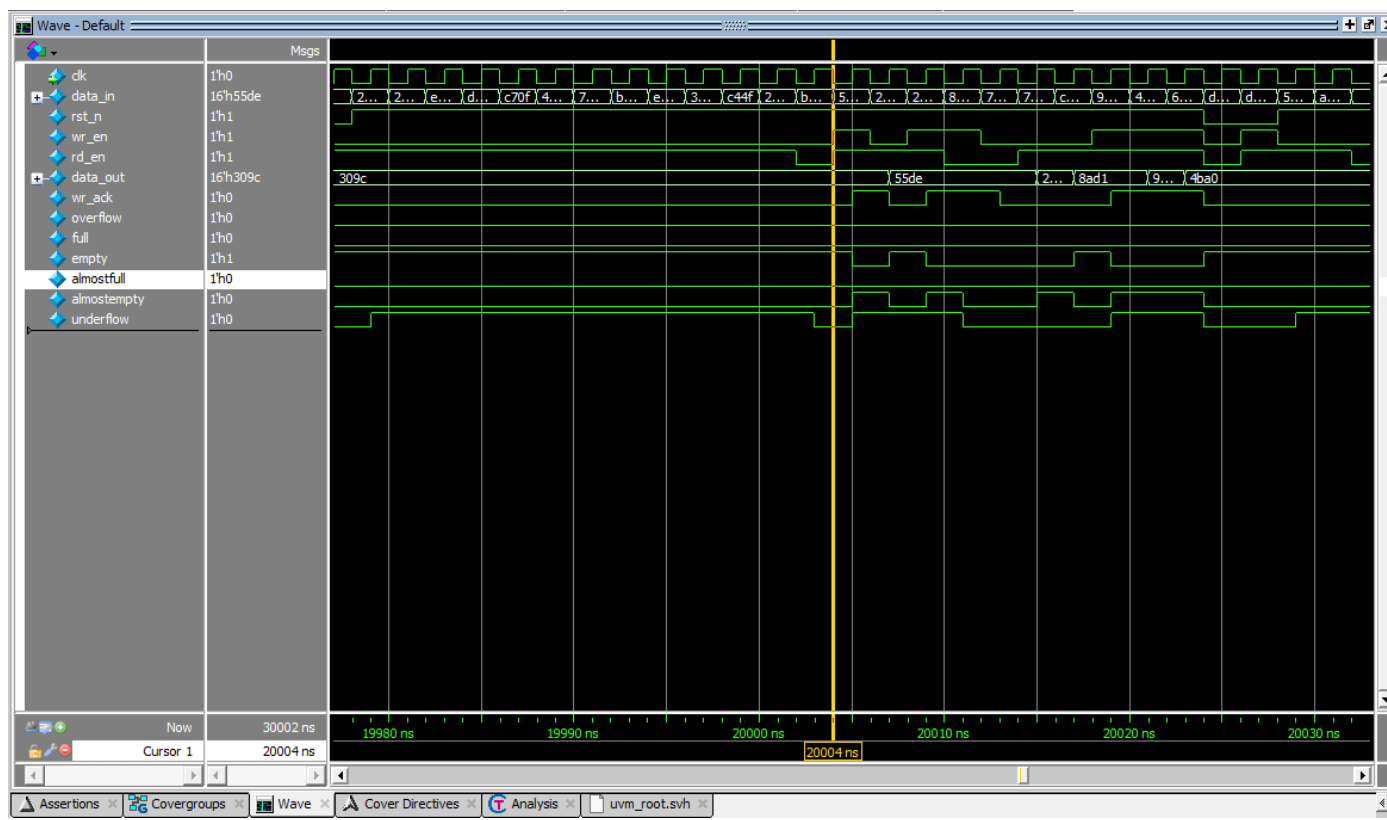
Writing only :



Reading only:



Write and read :



Coverage snippets from Questasim :

Cover Directives :

Cover Directives														
Name	Language	Enabled	Log	Count	Atleast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Peak Memory Time	Cumulative Threads
 /top/DUT/svas/reset_cp	SVA		Off	1518	1	Unli...	1	100%			0	0	0 ns	0
 /top/DUT/svas/wr_ack_cp	SVA		Off	4276	1	Unli...	1	100%			0	0	0 ns	0
 /top/DUT/svas/overflow_cp	SVA		Off	1791	1	Unli...	1	100%			0	0	0 ns	0
 /top/DUT/svas/underflow_cp	SVA		Off	4648	1	Unli...	1	100%			0	0	0 ns	0

Assertions

Covergroups

uvm_root.svh

Wave

Cover Directives

Functional coverage :

Covergroups										
Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment	
/fifo_coverage_pk...		100.00%								
TYPE FIFOCg		100.00%	100	100.00...		✓			auto(1)	
CVP FIFOC...		100.00%	100	100.00...		✓				
CVP FIFOC...		100.00%	100	100.00...		✓				
CVP FIFOC...		100.00%	100	100.00...		✓				
CVP FIFOC...		100.00%	100	100.00...		✓				
CVP FIFOC...		100.00%	100	100.00...		✓				
CVP FIFOC...		100.00%	100	100.00...		✓				
CVP FIFOC...		100.00%	100	100.00...		✓				
CVP FIFOC...		100.00%	100	100.00...		✓				
CROSS FIF...		100.00%	100	100.00...		✓				
CROSS FIF...		100.00%	100	100.00...		✓				
CROSS FIF...		100.00%	100	100.00...		✓				
CROSS FIF...		100.00%	100	100.00...		✓				
CROSS FIF...		100.00%	100	100.00...		✓				
CROSS FIF...		100.00%	100	100.00...		✓				
CROSS FIF...		100.00%	100	100.00...		✓				

Assertion coverage :

Assertions													
Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time	Cumulative Threads	ATV	Assertion Expression	
/lvm_pkg:uvm_reg_map::do_write/#ublk#215181159#1731/imm...	Immediate	SVA	on	0	0	-	-	-	-	-	off	assert (\$cast(seq,o))	
/lvm_pkg:uvm_reg_map::do_read/#ublk#215181159#1771/imm...	Immediate	SVA	on	0	0	-	-	-	-	-	off	assert (\$cast(seq,o))	
/fifo_write_read_seq_pkg::fifo_write_read_seq::body/#ublk#164...	Immediate	SVA	on	0	1	-	-	-	-	-	off	assert (/randomize(...))	
/fifo_read_seq_pkg::fifo_read_seq::body/#ublk#253944871#16/...	Immediate	SVA	on	0	1	-	-	-	-	-	off	assert (/randomize(...))	
/fifo_write_seq_pkg::fifo_write_seq::body/#ublk#26928919#16/...	Immediate	SVA	on	0	1	-	-	-	-	-	off	assert (/randomize(...))	
/top/DUT/svas/reset_ap	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert! @(posedge fifoif.	
/top/DUT/svas/wir_ack_ap	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert! @(posedge fifoif.	
/top/DUT/svas/overflow_ap	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert! @(posedge fifoif.	
/top/DUT/svas/underflow_ap	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert! @(posedge fifoif.	
/top/DUT/svas/full_flag_ap	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert! @(posedge fifoif.	
/top/DUT/svas/empty_flag_ap	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert! @(posedge fifoif.	
/top/DUT/svas/almostfull_ap	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert! @(posedge fifoif.	
/top/DUT/svas/almostempty_ap	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert! @(posedge fifoif.	
/top/DUT/svas/read_valid_ap	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert! @(posedge fifoif.	
/top/DUT/svas/write_valid_ap	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert! @(posedge fifoif.	

Coverage report :

Assertion report:

# =====				
# === Instance: /top/DUT/svas				
# === Design Unit: work.FIFO_sva				
# =====				
#				
# Assertion Coverage:				
#	Assertions	10	10	0 100.00%
# -----				
#	Name	File (Line)	Failure	Pass
#			Count	Count
# -----				
#	/top/DUT/svas/reset_ap			
#		FIFO_sva.sv(16)	0	1
#	/top/DUT/svas/wr_ack_ap			
#		FIFO_sva.sv(28)	0	1
#	/top/DUT/svas/overflow_ap			
#		FIFO_sva.sv(40)	0	1
#	/top/DUT/svas/underflow_ap			
#		FIFO_sva.sv(52)	0	1
#	/top/DUT/svas/full_flag_ap			
#		FIFO_sva.sv(64)	0	1
#	/top/DUT/svas/empty_flag_ap			
#		FIFO_sva.sv(75)	0	1
#	/top/DUT/svas/almostfull_ap			
#		FIFO_sva.sv(86)	0	1
#	/top/DUT/svas/almostempty_ap			
#		FIFO_sva.sv(97)	0	1
#	/top/DUT/svas/read_valid_ap			
#		FIFO_sva.sv(108)	0	1
#	/top/DUT/svas/write_valid_ap			
#		FIFO_sva.sv(119)	0	1
#				
# =====				

```
# === Instance: /fifo_write_read_seq_pkg
# === Design Unit: work.fifo_write_read_seq_pkg
# =====
#
# Assertion Coverage:
#   Assertions           1           1           0   100.00%
# -----
# Name                   File(Line)                   Failure    Pass
#                                     Count           Count
# -----
# /fifo_write_read_seq_pkg/fifo_write_read_seq/body/#ublk#164226423#16/immed__18
#   read_write_seq.sv(18)                                0           1
#
# =====
# === Instance: /fifo_read_seq_pkg
# === Design Unit: work.fifo_read_seq_pkg
# =====
#
# Assertion Coverage:
#   Assertions           1           1           0   100.00%
# -----
# Name                   File(Line)                   Failure    Pass
#                                     Count           Count
# -----
# /fifo_read_seq_pkg/fifo_read_seq/body/#ublk#253944871#16/immed__18
#   read_seq.sv(18)                                       0           1
#
# =====
# === Instance: /fifo_write_seq_pkg
# === Design Unit: work.fifo_write_seq_pkg
# =====
#
# Assertion Coverage:
#   Assertions           1           1           0   100.00%
# -----
# Name                   File(Line)                   Failure    Pass
#                                     Count           Count
# -----
# /fifo_write_seq_pkg/fifo_write_seq/body/#ublk#26928919#16/immed__18
```

```
# =====
# === Instance: /fifo_write_seq_pkg
# === Design Unit: work.fifo_write_seq_pkg
# =====
#
# Assertion Coverage:
#   Assertions           1           1           0   100.00%
# -----
# Name                   File(Line)                   Failure   Pass
#                   Count                               Count
# -----
# /fifo_write_seq_pkg/fifo_write_seq/body/#ublk#26928919#16/immed__18
#                   write_seq.sv(18)                   0         1
#
#
# ASSERTION RESULTS:
# -----
# Name                   File(Line)                   Failure   Pass
#                   Count                               Count
# -----
# /top/DUT/svas/reset_ap
#                   FIFO_sva.sv(16)                   0         1
# /top/DUT/svas/wr_ack_ap
#                   FIFO_sva.sv(28)                   0         1
# /top/DUT/svas/overflow_ap
#                   FIFO_sva.sv(40)                   0         1
# /top/DUT/svas/underflow_ap
#                   FIFO_sva.sv(52)                   0         1
# /top/DUT/svas/full_flag_ap
#                   FIFO_sva.sv(64)                   0         1
# /top/DUT/svas/empty_flag_ap
#                   FIFO_sva.sv(75)                   0         1
# /top/DUT/svas/almostfull_ap
#                   FIFO_sva.sv(86)                   0         1
# /top/DUT/svas/almostempty_ap
#                   FIFO_sva.sv(97)                   0         1
# /top/DUT/svas/read_valid_ap
#                   FIFO_sva.sv(108)                  0         1
# /top/DUT/svas/write_valid_ap
#                   FIFO_sva.sv(119)                  0         1
# /top/DUT/svas/read_valid_ap
#                   FIFO_sva.sv(108)                  0         1
# /top/DUT/svas/write_valid_ap
#                   FIFO_sva.sv(119)                  0         1
# /fifo_write_read_seq_pkg/fifo_write_read_seq/body/#ublk#164226423#16/immed__18
#                   read_write_seq.sv(18)             0         1
# /fifo_read_seq_pkg/fifo_read_seq/body/#ublk#253944871#16/immed__18
#                   read_seq.sv(18)                   0         1
# /fifo_write_seq_pkg/fifo_write_seq/body/#ublk#26928919#16/immed__18
#                   write_seq.sv(18)                   0         1
#
#
# Total Coverage By Instance (filtered view): 100.00%
#
#
```

Code coverage report :

```

Branch Coverage:
  Enabled Coverage      Bins      Hits      Misses      Coverage
  -----
  Branches              25       25         0      100.00%

=====Branch Details=====

Branch Coverage for instance /top/DUT

  Line      Item      Count      Source
  ----      -
  File FIFO.sv
  -----IF Branch-----
  10              16355      Count coming in to IF
  10              2872      if (!fifoif.rst_n) begin
  15              4744      else if (fifoif.wr_en && count < FIFO_DEPTH) begin
  21              8739      else begin
Branch totals: 3 hits of 3 branches = 100.00%

  -----IF Branch-----
  23              8739      Count coming in to IF
  23              1986      if (fifoif.full && fifoif.wr_en) // & must be &&
  25              6753      else
Branch totals: 2 hits of 2 branches = 100.00%

  -----IF Branch-----
  31              16355      Count coming in to IF
  31              2872      if (!fifoif.rst_n) begin
  36              1499      else if (fifoif.rd_en && count != 0) begin
  41              11984      else begin
Branch totals: 3 hits of 3 branches = 100.00%

  -----IF Branch-----
  42              11984      Count coming in to IF
  42              5165      if(fifoif.empty && fifoif.rd_en)begin
  45              6819      else begin
Branch totals: 2 hits of 2 branches = 100.00%

  -----IF Branch-----
  53              9898      Count coming in to IF
  53              2757      if (!fifoif.rst_n) begin
  56              7141      else begin
Branch totals: 2 hits of 2 branches = 100.00%

  -----IF Branch-----
  57              7141      Count coming in to IF
  57              3662      if ( ({fifoif.wr_en, fifoif.rd_en} == 2'b10) && !fifoif.full)
  59              760      else if ( ({fifoif.wr_en, fifoif.rd_en} == 2'b01) && !fifoif.empty)
  61              351      else if ( ({fifoif.wr_en, fifoif.rd_en} == 2'b11) && fifoif.empty)
//missing case
  63              8      else if ( ({fifoif.wr_en, fifoif.rd_en} == 2'b11) && fifoif.full)
              2360      All False Count
Branch totals: 5 hits of 5 branches = 100.00%

  -----IF Branch-----
  68              5566      Count coming in to IF
  68              234      assign fifoif.full = (count == FIFO_DEPTH)? 1 : 0;
  68              5332      assign fifoif.full = (count == FIFO_DEPTH)? 1 : 0;
Branch totals: 2 hits of 2 branches = 100.00%

```



```
-----IF Branch-----
69          5566      Count coming in to IF
69          1      1149      assign fifoif.empty = (count == 0)? 1 : 0;
69          2      4417      assign fifoif.empty = (count == 0)? 1 : 0;
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
72          5566      Count coming in to IF
72          1      263      assign fifoif.almostfull = (count == FIFO_DEPTH-1)? 1 : 0; //bug in -2
72          2      5303      assign fifoif.almostfull = (count == FIFO_DEPTH-1)? 1 : 0; //bug in -2
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
73          5566      Count coming in to IF
73          1      1333      assign fifoif.almostempty = (count == 1)? 1 : 0;
73          2      4233      assign fifoif.almostempty = (count == 1)? 1 : 0;
Branch totals: 2 hits of 2 branches = 100.00%
```

Condition Coverage:

Enabled Coverage	Bins	Covered	Misses	Coverage
-----	-----	-----	-----	-----
Conditions	24	24	0	100.00%

====Condition Details=====

Condition Coverage for instance /top/DUT --

File FIFO.sv

-----Focused Condition View-----

Line 15 Item 1 (fifoif.wr_en && (count < 8))

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
-----	-----	-----	-----
fifoif.wr_en	Y		
(count < 8)	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
-----	-----	-----	-----
Row 1:	1	fifoif.wr_en_0	-
Row 2:	1	fifoif.wr_en_1	(count < 8)
Row 3:	1	(count < 8)_0	fifoif.wr_en
Row 4:	1	(count < 8)_1	fifoif.wr_en

-----Focused Condition View-----

Line 23 Item 1 (fifoif.full && fifoif.wr_en)

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
-----	-----	-----	-----
fifoif.full	Y		
fifoif.wr_en	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
-----	-----	-----	-----
Row 1:	1	fifoif.full_0	-
Row 2:	1	fifoif.full_1	fifoif.wr_en
Row 3:	1	fifoif.wr_en_0	fifoif.full

```
Row 2:      1  fifoif.full_1  fifoif.wr_en
Row 3:      1  fifoif.wr_en_0  fifoif.full
Row 4:      1  fifoif.wr_en_1  fifoif.full
```

-----Focused Condition View-----

Line 36 Item 1 (fifoif.rd_en && (count != 0))

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
-----	-----	-----	-----
fifoif.rd_en	Y		
(count != 0)	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
-----	-----	-----	-----
Row 1:	1	fifoif.rd_en_0	-
Row 2:	1	fifoif.rd_en_1	(count != 0)
Row 3:	1	(count != 0)_0	fifoif.rd_en
Row 4:	1	(count != 0)_1	fifoif.rd_en

-----Focused Condition View-----

Line 42 Item 1 (fifoif.empty && fifoif.rd_en)

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
-----	-----	-----	-----
fifoif.empty	Y		
fifoif.rd_en	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
-----	-----	-----	-----
Row 1:	1	fifoif.empty_0	-
Row 2:	1	fifoif.empty_1	fifoif.rd_en
Row 3:	1	fifoif.rd_en_0	fifoif.empty
Row 4:	1	fifoif.rd_en_1	fifoif.empty

-----Focused Condition View-----

Line 57 Item 1 ((~fifoif.rd_en && fifoif.wr_en) && ~fifoif.full)

Condition totals: 3 of 3 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
-----	-----	-----	-----
fifoif.rd_en	Y		
fifoif.wr_en	Y		
fifoif.full	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
-----	-----	-----	-----
Row 1:	1	fifoif.rd_en_0	(~fifoif.full && fifoif.wr_en)
Row 2:	1	fifoif.rd_en_1	-
Row 3:	1	fifoif.wr_en_0	~fifoif.rd_en
Row 4:	1	fifoif.wr_en_1	(~fifoif.full && ~fifoif.rd_en)
Row 5:	1	fifoif.full_0	(~fifoif.rd_en && fifoif.wr_en)
Row 6:	1	fifoif.full_1	(~fifoif.rd_en && fifoif.wr_en)

-----Focused Condition View-----

Line 59 Item 1 ((fifoif.rd_en && ~fifoif.wr_en) && ~fifoif.empty)

Condition totals: 3 of 3 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
-----	-----	-----	-----
fifoif.rd_en	Y		
fifoif.wr_en	Y		
fifoif.empty	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	fifoif.rd_en_0	-
Row 2:	1	fifoif.rd_en_1	(~fifoif.empty && ~fifoif.wr_en)
Row 3:	1	fifoif.wr_en_0	(~fifoif.empty && fifoif.rd_en)
Row 4:	1	fifoif.wr_en_1	fifoif.rd_en
Row 5:	1	fifoif.empty_0	(fifoif.rd_en && ~fifoif.wr_en)
Row 6:	1	fifoif.empty_1	(fifoif.rd_en && ~fifoif.wr_en)

-----Focused Condition View-----

Line 61 Item 1 ((fifoif.rd_en && fifoif.wr_en) && fifoif.empty)
Condition totals: 3 of 3 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
fifoif.rd_en	Y		
fifoif.wr_en	Y		
fifoif.empty	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	fifoif.rd_en_0	-
Row 2:	1	fifoif.rd_en_1	(fifoif.empty && fifoif.wr_en)
Row 3:	1	fifoif.wr_en_0	fifoif.rd_en
Row 4:	1	fifoif.wr_en_1	(fifoif.empty && fifoif.rd_en)
Row 5:	1	fifoif.empty_0	(fifoif.rd_en && fifoif.wr_en)
Row 6:	1	fifoif.empty_1	(fifoif.rd_en && fifoif.wr_en)

-----Focused Condition View-----

Line 63 Item 1 ((fifoif.rd_en && fifoif.wr_en) && fifoif.full)
Condition totals: 3 of 3 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
fifoif.rd_en	Y		
fifoif.wr_en	Y		
fifoif.full	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	fifoif.rd_en_0	-
Row 2:	1	fifoif.rd_en_1	(fifoif.full && fifoif.wr_en)
Row 3:	1	fifoif.wr_en_0	fifoif.rd_en
Row 4:	1	fifoif.wr_en_1	(fifoif.full && fifoif.rd_en)
Row 5:	1	fifoif.full_0	(fifoif.rd_en && fifoif.wr_en)
Row 6:	1	fifoif.full_1	(fifoif.rd_en && fifoif.wr_en)

-----Focused Condition View-----

Line 68 Item 1 (count == 8)
Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
(count == 8)	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(count == 8)_0	-
Row 2:	1	(count == 8)_1	-

-----Focused Condition View-----

Line 69 Item 1 (count == 0)
Condition totals: 1 of 1 input term covered = 100.00%

Input Term		Covered	Reason for no coverage	Hint
(count == (8 - 1))		Y		
Rows:		Hits	FEC Target	Non-masking condition(s)
Row 1:		1	(count == (8 - 1))_0	-
Row 2:		1	(count == (8 - 1))_1	-
-----Focused Condition View-----				
Line	73 Item	1	(count == 1)	
Condition totals: 1 of 1 input term covered = 100.00%				
Input Term		Covered	Reason for no coverage	Hint
(count == 1)		Y		
Rows:		Hits	FEC Target	Non-masking condition(s)
Row 1:		1	(count == 1)_0	-
Row 2:		1	(count == 1)_1	-
Statement Coverage:				
Enabled Coverage		Bins	Hits	Misses Coverage
-----		----	----	-----
Statements		27	27	0 100.00%

=====Statement Details=====

Statement Coverage for instance /top/DUT --

Line ----	Item ----	Count -----	Source -----
File FIFO.sv			
1			module FIFO(FIFO_if,DUT fifoif);
2			parameter FIFO_WIDTH = 16;
3			parameter FIFO_DEPTH = 8;
4			localparam max_fifo_addr = \$clog2(FIFO_DEPTH);
5			reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];
6			reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
7			reg [max_fifo_addr:0] count;
8			
9	1	16355	always @(posedge fifoif.clk or negedge fifoif.rst_n) begin
10			if (!fifoif.rst_n) begin
11	1	2872	wr_ptr <= 0;
12	1	2872	fifoif.wr_ack <= 0; //missing after reset
13	1	2872	fifoif.overflow <= 0; //missing after reset
14			end
15			else if (fifoif.wr_en && count < FIFO_DEPTH) begin
16	1	4744	mem[wr_ptr] <= fifoif.data_in;
17	1	4744	fifoif.wr_ack <= 1;
18	1	4744	wr_ptr <= wr_ptr + 1;
19			
20			end
21			else begin
22	1	8739	fifoif.wr_ack <= 0;
23			if (fifoif.full && fifoif.wr_en) // & must be &&
24	1	1986	fifoif.overflow <= 1;
25			else
26	1	6753	fifoif.overflow <= 0;
27			end
28			end
29			
30	1	16355	always @(posedge fifoif.clk or negedge fifoif.rst_n) begin
31			if (!fifoif.rst_n) begin
32	1	2872	rd_ptr <= 0;
33	1	2872	fifoif.underflow <= 0; //missing after reset
34			
35			end
36			else if (fifoif.rd_en && count != 0) begin
37	1	1499	fifoif.data_out <= mem[rd_ptr];
38	1	1499	rd_ptr <= rd_ptr + 1;
39			
40			end
41			else begin
42			if(fifoif.empty && fifoif.rd_en)begin
43	1	5165	fifoif.underflow <= 1;
44			end
45			else begin
46	1	6819	fifoif.underflow <= 0;
47			end
48			end
49			end
50			
51			
52	1	9898	always @(posedge fifoif.clk or negedge fifoif.rst_n) begin
53			if (!fifoif.rst_n) begin

```

52      1          9898      always @(posedge fifoif.clk or negedge fifoif.rst_n) begin
53          if (!fifoif.rst_n) begin
54              1          2757          count <= 0;
55          end
56          else begin
57              if      ( ({fifoif.wr_en, fifoif.rd_en} == 2'b10) && !fifoif.full)
58                  1          3662          count <= count + 1;
59              else if ( ({fifoif.wr_en, fifoif.rd_en} == 2'b01) && !fifoif.empty)
60                  1          760          count <= count - 1;
61              else if ( ({fifoif.wr_en, fifoif.rd_en} == 2'b11) && fifoif.empty)
//missing case
62                  1          351          count <= count + 1;
63              else if ( ({fifoif.wr_en, fifoif.rd_en} == 2'b11) && fifoif.full)
64                  1          8          count <= count - 1;
65          end
66      end
67
68      1          5567      assign fifoif.full = (count == FIFO_DEPTH)? 1 : 0;
69      1          5567      assign fifoif.empty = (count == 0)? 1 : 0;
70      //assign fifoif.underflow = (fifoif.empty && fifoif.rd_en)? 1 : 0;    // its
mentioned in pdf its a sequential
71
72      1          5567      assign fifoif.almostfull = (count == FIFO_DEPTH-1)? 1 : 0; //bug in -2
73      1          5567      assign fifoif.almostempty = (count == 1)? 1 : 0;

```

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	20	20	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /top/DUT --

Node	1H->0L	0L->1H	"Coverage"
count[3-0]	1	1	100.00
rd_ptr[2-0]	1	1	100.00
wr_ptr[2-0]	1	1	100.00

Total Node Count = 10
 Toggled Node Count = 10
 Untoggled Node Count = 0
 Toggle Coverage = 100.00% (20 of 20 bins)

Functional coverage report:

Covergroup Coverage:					
Covergroups	1	na	na	100.00%	
Coverpoints/Crosses	16	na	na	na	
Covergroup Bins	66	66	0	100.00%	

Covergroup		Metric	Goal	Bins	Status

TYPE /fifo_coverage_pkg/fifo_coverage/FIFOcg		100.00%	100	-	Covered
covered/total bins:		66	66	-	
missing/total bins:		0	66	-	
% Hit:		100.00%	100	-	
Coverpoint wr_en_cp		100.00%	100	-	Covered
covered/total bins:		2	2	-	
missing/total bins:		0	2	-	
% Hit:		100.00%	100	-	
bin auto[0]		7524	1	-	Covered
bin auto[1]		7476	1	-	Covered
Coverpoint rd_en_cp		100.00%	100	-	Covered
covered/total bins:		2	2	-	
missing/total bins:		0	2	-	
% Hit:		100.00%	100	-	
bin auto[0]		7570	1	-	Covered
bin auto[1]		7431	1	-	Covered
Coverpoint full_cp		100.00%	100	-	Covered
covered/total bins:		2	2	-	
missing/total bins:		0	2	-	
% Hit:		100.00%	100	-	
bin auto[0]		12781	1	-	Covered
bin auto[1]		2220	1	-	Covered
Coverpoint almostfull_cp		100.00%	100	-	Covered
covered/total bins:		2	2	-	
missing/total bins:		0	2	-	
% Hit:		100.00%	100	-	
bin auto[0]		14714	1	-	Covered
bin auto[1]		287	1	-	Covered
Coverpoint empty_cp		100.00%	100	-	Covered
covered/total bins:		2	2	-	
missing/total bins:		0	2	-	
% Hit:		100.00%	100	-	
bin auto[0]		7919	1	-	Covered
bin auto[1]		7082	1	-	Covered
Coverpoint almostempty_cp		100.00%	100	-	Covered
covered/total bins:		2	2	-	
missing/total bins:		0	2	-	
% Hit:		100.00%	100	-	
bin auto[0]		12947	1	-	Covered
bin auto[1]		2054	1	-	Covered
Coverpoint overflow_cp		100.00%	100	-	Covered
covered/total bins:		2	2	-	
missing/total bins:		0	2	-	
% Hit:		100.00%	100	-	
bin auto[0]		13010	1	-	Covered
bin auto[1]		1991	1	-	Covered
Coverpoint underflow_cp		100.00%	100	-	Covered
covered/total bins:		2	2	-	
missing/total bins:		0	2	-	
% Hit:		100.00%	100	-	
bin auto[0]		9640	1	-	Covered
bin auto[1]		5361	1	-	Covered
Coverpoint wr_ack_cp		100.00%	100	-	Covered
covered/total bins:		2	2	-	
missing/total bins:		0	2	-	

```

missing/total bins: 100.00% 100 -
% Hit: 100.00% 100 -
bin auto[0] 10257 1 - Covered
bin auto[1] 4744 1 - Covered
Cross full_cross 100.00% 100 - Covered
covered/total bins: 6 6 -
missing/total bins: 0 6 -
% Hit: 100.00% 100 -
Auto, Default and User Defined Bins:
bin <auto[1],auto[1],auto[0]> 1211 1 - Covered
bin <auto[0],auto[1],auto[0]> 6220 1 - Covered
bin <auto[1],auto[0],auto[1]> 2212 1 - Covered
bin <auto[1],auto[0],auto[0]> 4053 1 - Covered
bin <auto[0],auto[0],auto[1]> 8 1 - Covered
bin <auto[0],auto[0],auto[0]> 1296 1 - Covered
Illegal and Ignore Bins:
ignore_bin rd_full 0 - ZERO
Cross almostfull_cross 100.00% 100 - Covered
covered/total bins: 8 8 -
missing/total bins: 0 8 -
% Hit: 100.00% 100 -
Auto, Default and User Defined Bins:
bin <auto[1],auto[1],auto[1]> 20 1 - Covered
bin <auto[0],auto[1],auto[1]> 6 1 - Covered
bin <auto[1],auto[0],auto[1]> 249 1 - Covered
bin <auto[0],auto[0],auto[1]> 12 1 - Covered
bin <auto[1],auto[1],auto[0]> 1191 1 - Covered
bin <auto[0],auto[1],auto[0]> 6214 1 - Covered
bin <auto[1],auto[0],auto[0]> 6016 1 - Covered
bin <auto[0],auto[0],auto[0]> 1292 1 - Covered
Cross empty_cross 100.00% 100 - Covered
covered/total bins: 8 8 -
missing/total bins: 0 8 -
% Hit: 100.00% 100 -
Auto, Default and User Defined Bins:
bin <auto[1],auto[1],auto[1]> 121 1 - Covered
bin <auto[0],auto[1],auto[1]> 5824 1 - Covered
bin <auto[1],auto[0],auto[1]> 625 1 - Covered
bin <auto[0],auto[0],auto[1]> 511 1 - Covered
bin <auto[1],auto[1],auto[0]> 1090 1 - Covered
bin <auto[0],auto[1],auto[0]> 396 1 - Covered
bin <auto[1],auto[0],auto[0]> 5640 1 - Covered
bin <auto[0],auto[0],auto[0]> 793 1 - Covered
Cross almostempty_cross 100.00% 100 - Covered
covered/total bins: 8 8 -
missing/total bins: 0 8 -
% Hit: 100.00% 100 -
Auto, Default and User Defined Bins:
bin <auto[1],auto[1],auto[1]> 702 1 - Covered
bin <auto[0],auto[1],auto[1]> 184 1 - Covered
bin <auto[1],auto[0],auto[1]> 798 1 - Covered
bin <auto[0],auto[0],auto[1]> 370 1 - Covered
bin <auto[1],auto[1],auto[0]> 509 1 - Covered
bin <auto[0],auto[1],auto[0]> 6036 1 - Covered
bin <auto[1],auto[0],auto[0]> 5467 1 - Covered
bin <auto[0],auto[0],auto[0]> 934 1 - Covered
Cross overflow_cross 100.00% 100 - Covered
covered/total bins: 6 6 -
missing/total bins: 0 6 -
% Hit: 100.00% 100 -
Auto, Default and User Defined Bins:
bin <auto[1],auto[1],auto[1]> 9 1 - Covered
bin <auto[1],auto[0],auto[1]> 1982 1 - Covered
bin <auto[1],auto[1],auto[0]> 1202 1 - Covered
bin <auto[0],auto[1],auto[0]> 6220 1 - Covered
bin <auto[1],auto[0],auto[0]> 4283 1 - Covered
bin <auto[0],auto[0],auto[0]> 1304 1 - Covered

```



```

    bin <auto[0],auto[0],auto[0]>          1304          1          -          Covered
Illegal and Ignore Bins:
    ignore_bin not_wr_overflow              0              -          ZERO
Cross underflow_cross                    100.00%         100          -          Covered
    covered/total bins:                     6              6          -
    missing/total bins:                     0              6          -
    % Hit:                                100.00%         100          -
Auto, Default and User Defined Bins:
    bin <auto[1],auto[1],auto[1]>          454              1          -          Covered
    bin <auto[1],auto[1],auto[0]>          757              1          -          Covered
    bin <auto[0],auto[1],auto[1]>          4907             1          -          Covered
    bin <auto[0],auto[1],auto[0]>          1313             1          -          Covered
    bin <auto[1],auto[0],auto[0]>          6265             1          -          Covered
    bin <auto[0],auto[0],auto[0]>          1304             1          -          Covered
Illegal and Ignore Bins:
    ignore_bin not_rd_overflow              0              -          ZERO
Cross wr_ack_cross                      100.00%         100          -          Covered
    covered/total bins:                     6              6          -
    missing/total bins:                     0              6          -
    % Hit:                                100.00%         100          -
Auto, Default and User Defined Bins:
    bin <auto[1],auto[1],auto[1]>          1082             1          -          Covered
    bin <auto[1],auto[0],auto[1]>          3662             1          -          Covered
    bin <auto[1],auto[1],auto[0]>           129             1          -          Covered
    bin <auto[0],auto[1],auto[0]>          6220             1          -          Covered
    bin <auto[1],auto[0],auto[0]>          2603             1          -          Covered
    bin <auto[0],auto[0],auto[0]>          1304             1          -          Covered
Illegal and Ignore Bins:
    ignore_bin not_wren_wrack              0              -          ZERO

```

Assertions Table:

2		
3	Feature	Assertion
4	During reset, wr_ack, overflow, and	`@(posedge clk) (!rst_n)
5	underflow must be 0	
6	When write is enabled and FIFO is not	`@(posedge clk) (!full &&
7	full, wr_ack must be asserted	wr_en)
8	When FIFO is full and write is enabled,	`@(posedge clk) (full &&
9	overflow must be asserted	wr_en)
10	When FIFO is empty and read is enabled,	`@(posedge clk) (empty
11	underflow must be asserted	&& rd_en)
12	Full flag should remain asserted when	`@(posedge clk) full
13	FIFO is full	
14	Empty flag should remain asserted when	`@(posedge clk) empty
15	FIFO is empty	
16	Almost full flag should be asserted	`@(posedge clk) almostfull
17	correctly	
18	Almost empty flag should be asserted	`@(posedge clk)
19	correctly	almostempty
20	When read is valid, data_out should not	`@(posedge clk) (rd_en &&
21	be unknown	!empty)
22	When write is valid, wr_ack must be	`@(posedge clk) (wr_en &&
	asserted	!full)